CMSC 420: Blockchains

Justin Wyss-Gallifent

January 21, 2026

1 Problems **2**

1 Problems

1. The following is a really basic (and pretty terrible) simple version of SHA-256. It only works on keys M which are binary strings whose length is a multiple of 4.

   (a) Break M up into N sub-keys of length 4 bits: $M^{(1)}$, $M^{(2)}$, ... , $M^{(N)}$.

   (b) Initialize $H = \texttt{0101}$ and $K = \texttt{0111}$.

   (c) For each sub-key $M^{(i)}$ do the following:

      i. Update H by XORing it with $M^{(i)}$.

      ii. Update H by XORing it with K.

      iii. Left-rotate K by 1 bit.

   (d) The output is the resulting H, so $h(M) = H$.

   (a) Calculate $h(\texttt{11110100})$. Show the value of H after each step.
   Solution:
   We have: $\texttt{0101} \rightarrow \texttt{1101} \rightarrow \texttt{0111}$
   The result is 0111.

   (b) Calculate $h(\texttt{00000000})$. Show the value of H after each step.
   Solution:
   We have: $\texttt{0101} \rightarrow \texttt{0010} \rightarrow \texttt{1100}$
   The result is 1100.

   (c) Find the smallest 8-bit nonce which can be prepended to 10101010 so that the output starts with at exactly two 0s. Do this any way you wish but justify your approach.
   Solution:
   If the nonce is 00001000 then the message is 0000100010101010 and H progresses as follows:
   $\texttt{0101} \rightarrow \texttt{0010} \rightarrow \texttt{0100} \rightarrow \texttt{0011} \rightarrow \texttt{0010}$
   The result is 0010.
   I did this by writing Python code to do the hashing and then trial-and-error.

2. In this problem we construct a small blockchain using the hash function from the previous question. We won't worry about including the date.

   (a) The first block contains the message `LOVE`. Convert this to ASCII binary. This is our first block.

   Solution:
   We convert to get the message:

   $$\texttt{LOVE} \longrightarrow \texttt{01001100010011110101011001000101}$$

   This is our first block.

   (b) The second block contains the message `YOU`. Convert this to ASCII binary, hash the first block, append that hash to the message. This is our second block.

   Solution:
   We convert to get the message:

   $$\texttt{YOU} \longrightarrow \texttt{010110010100111101010101}$$

   We hash the first block:

   $$h(\texttt{01001100010011110101011001000101}) = \texttt{0100}$$

   We append the hash to the message to get:

   $$\texttt{01011001010011110101010101 00}$$

   This is our second block.

   (c) The third block contains the message `CS`. Convert this to ASCII binary, hash the second block, append that hash to the message. This is our third block.

   Solution:
   We convert to get the message:

   $$\texttt{CS} \longrightarrow \texttt{0100001101010011}$$

   We hash the second block:

   $$h(\texttt{0101100101001111010101010100}) = \texttt{1101}$$

   We append the hash to the message to get:

   $$\texttt{01000011010100111101}$$

3. Suppose we have the following set of transactions. Note that they don't all have the same number of bits, which is not a problem:

$$111111, 010101010, 110100000, 001011010, 111011, 010$$

Use the very simple hash function h which breaks a binary string into chunks of 3 bits and XORs them to create the Merkle tree for this set of transactions.

Solution: The leaves of the Merkle tree will be the hashes of these keys with no need for duplication since there is an even number, hence:

$$000, 101, 010, 101, 100, 010$$

We concatenate pairs and hash again with the last one duplicated since there is an odd number, hence:

$$101, 111, 110, 110$$

We concatenate pairs and hash again with no need for duplication since there is an even number, hence:

$$010, 000$$

We concatenate pairs and hash again, no need for duplication since there is an even number, hence:

$$010$$

Thus the Merkle tree is:

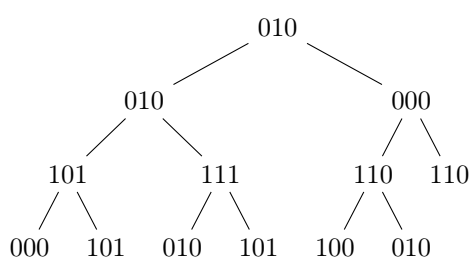

Figure 1: Merkle Tree

4. Suppose have the following Merkle tree which was created using the very simple hash function h which breaks a binary string into chunks of 3 bits and XORs them:

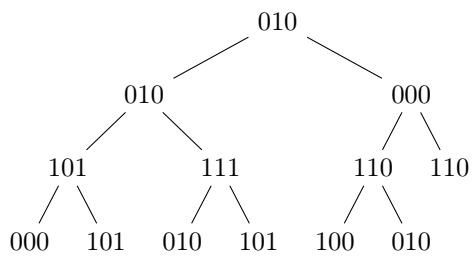

Figure 2: Merkle Tree

Provide a Merkle proof that the transaction 111011 is a valid transaction.

Solution:
Observe that $h(111011) = 100$. The proof can then be described with:

$$111011, (010, R), (110, R), (010, L)$$

5. One thing we glossed over is if two transactions have the same hash (very unlikely unless they're the same transaction) then the concept of a Merkle proof presents an issue. Explain.

   Solution:
   Suppose a block contains one transaction but another transaction which is not in the block has the same hash. Then a Merkle proof for the transaction which is in the block would also be a valid proof for the transaction which is not in the block. Note, however, that coming up with another valid transaction (not in the block) with the same hash essentially means finding a collision, and since hashes are difficult, if not impossible, to invert, this is not realistic.

6. Suppose a Merkle tree has n keys. For each of the following n, how many nodes will be in the Merkle tree?

n	Number of Nodes
5	13
6	13
2	3
5	13
17	25
11	41

7. Suppose a Merkle tree has n keys. If $f(n)$ is the number of nodes in the Merkle tree, prove that:

$$f(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\lceil n/2 \rceil + f(\lceil n/2 \rceil) & \text{otherwise} \end{cases}$$

Solution: If $n = 1$ there is clearly just the root, so $f(1) = 1$.

Otherwise, the number of nodes in the lowest level will be the smallest even integer greater than or equal to n, and this is $2\lceil n/2 \rceil$.

Above this since we pair these up we have a tree with $\lceil n/2 \rceil$ nodes in the lowest level which then has $f(\lceil n/2 \rceil)$ nodes total.

The result follows.